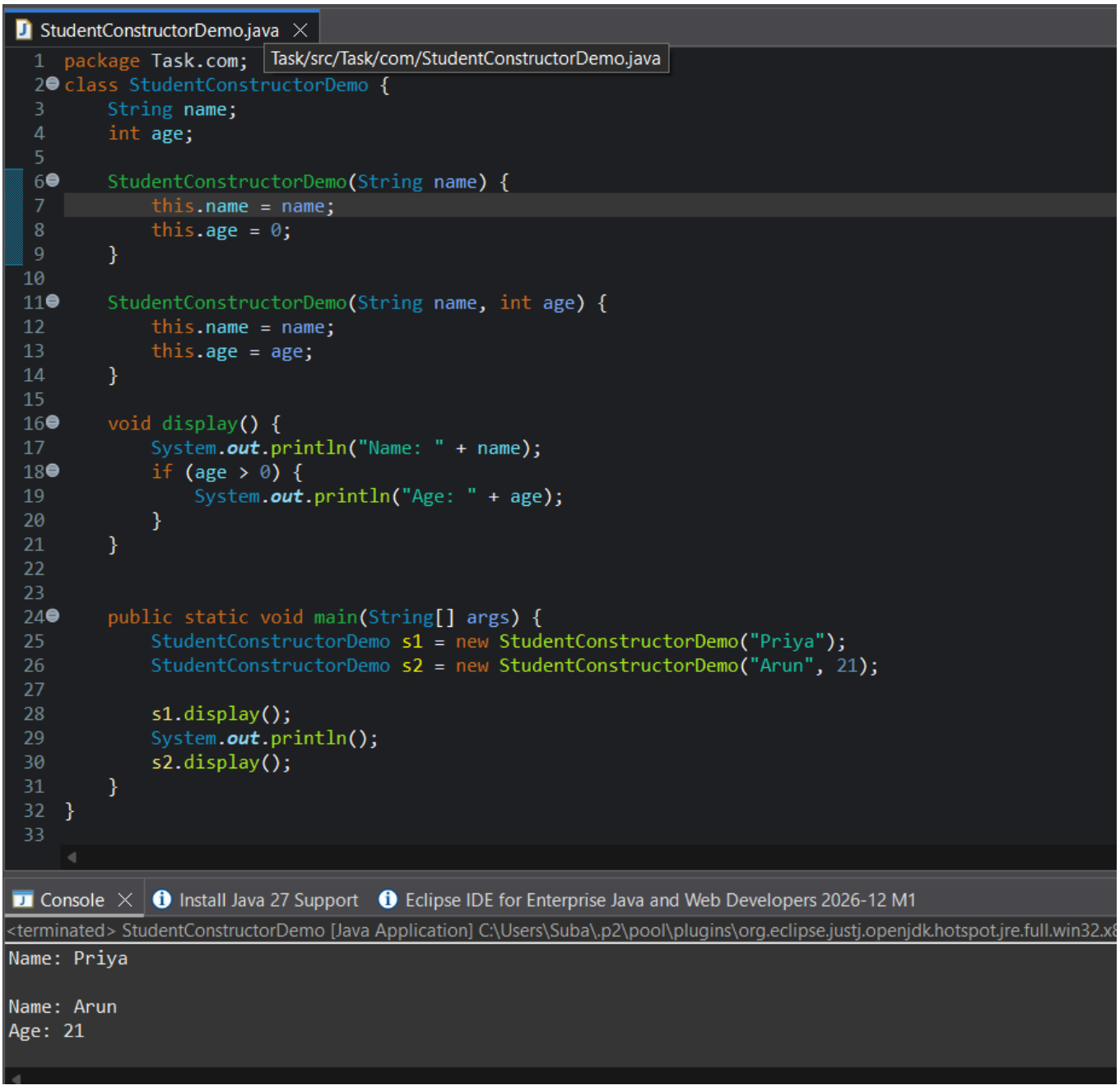
1. Constructor Overloading



2. Method Overloading and Method Overriding

StudentConstructorDemo.java | EmployeeDemo.java | MethodOverloadingOverriding.java

```java
package Task.com;

class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}

class Employee {
    double calculateBonus() {
        return 1000;
    }
}

class Developer extends Employee {
    public Developer() {
    }

    @Override
    double calculateBonus() {
        return 5000;
    }
}
```

```java
class Employee {
    double calculateBonus() {
        return 1000;
    }
}

class Developer extends Employee {
    public Developer() {
    }

    @Override
    double calculateBonus() {
        return 5000;
    }
}

public class MethodOverloadingOverriding {
    public static void main(String[] args) {
        Calculator c = new Calculator();

        System.out.println("Integer Addition: " + c.add(10, 20));
        System.out.println("Decimal Addition: " + c.add(10.5, 20.5));

        Employee e = new Developer();

        System.out.println("Employee Bonus: " + e.calculateBonus());
    }
}
```

Console | Install Java 27 Support | Eclipse IDE for Enterprise Java and Web Developers 2026-12 M1

<terminated> MethodOverloadingOverriding [Java Application] C:\Users\Suba\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.

```
Integer Addition: 30
Decimal Addition: 31.0
Employee Bonus: 5000.0
```

3. Encapsulation — Bank Account

```java
package Task.com;

class BankAccount {
    private String accountHolder;
    private double balance;

    BankAccount(String name, double balance) {
        this.accountHolder = name;
        this.balance = Math.max(0, balance);
    }

    public String getAccountHolder() {
        return accountHolder;
    }

    public double getBalance() {
        return balance;
    }

    public void setAccountHolder(String name) {
        if (name != null && !name.trim().isEmpty()) {
            accountHolder = name;
        }
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposit successful");
        } else {
            System.out.println("Invalid deposit amount");
        }
    }
```

```
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawal successful");
        } else {
            System.out.println("Insufficient balance or invalid amount");
        }
    }
}

public class BankAccountDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("Pinky", 5000);

        System.out.println("Account Holder: " + account.getAccountHolder());
        System.out.println("Balance: " + account.getBalance());

        account.withdraw(2000);
        System.out.println("Balance: " + account.getBalance());

        account.withdraw(4000);
        System.out.println("Balance: " + account.getBalance());
    }
}
```

Console

<terminated> BankAccountDemo [Java Application] C:\Users\Suba\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_25.0

```
Account Holder: Pinky
Balance: 5000.0
Withdrawal successful
Balance: 3000.0
Insufficient balance or invalid amount
Balance: 3000.0
```

4. Inheritance and Method Overriding

```java
package Task.com;

class CompanyEmployee {
    String name;
    double salary;

    CompanyEmployee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
    }

    void displayResponsibilities() {
        System.out.println("General employee responsibilities");
    }
}

class SoftwareDeveloper extends CompanyEmployee {

    SoftwareDeveloper(String name, double salary) {
        super(name, salary);
    }

    @Override
    void displayResponsibilities() {
        System.out.println(
            "Responsibility: Develop and maintain software"
        );
    }
}

class ProjectManager extends CompanyEmployee {

    ProjectManager(String name, double salary) {
        super(name, salary);
    }
```

```
Name: Arun
Salary: 40000.0
Responsibility: Develop and maintain software

Name: Priya
Salary: 60000.0
Responsibility: Manage the team and projects
```

5. Abstraction and Runtime Polymorphism

PaymentDemo.java

```java
package Task.com;

abstract class Payment {
    abstract void processPayment();
}

class UPIPayment extends Payment {
    @Override
    void processPayment() {
        System.out.println("Processing payment through UPI");
    }
}

class CreditCardPayment extends Payment {
    @Override
    void processPayment() {
        System.out.println("Processing payment through Credit Card");
    }
}

public class PaymentDemo {
    public static void main(String[] args) {
        Payment p;

        p = new UPIPayment();
        p.processPayment();

        p = new CreditCardPayment();
        p.processPayment();
    }
}
```

Console

```
<terminated> PaymentDemo [Java Application] C:\Users\Suba\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_25.0.4.v
Processing payment through UPI
Processing payment through Credit Card
```